Python Programming Lab. [Batch-B3]

Assignment Submission Details

Field	Details
Student Name *	Dilip Balu Magar
Student PRN *	125M1H055
Course Name	Python Programming Lab. (MCA31PC06)
Academic Year	2025-26 (Semester-1)
Course Teacher	Prof. Prakash Ukhalkar
Assignment Name	Assignment based on Exception Handling in Python 2.
Assignment Number	Assignment 09
Submission Date	27-11-2025

1.Create a Python library with custom exceptions for error handling.

In [1]:
```
%%writefile custom_exceptions.py
class PythonException(Exception):
    pass

class NegativeNumberError(Exception):
    pass

class InvalidFilePathError(Exception):
    pass

class InsufficientFundsError(Exception):
    pass

class InvalidPasswordError(Exception):
    pass
```

```
Writing custom_exceptions.py
```

In [2]:
```
from custom_exceptions import NegativeNumberError
try:
    num = int(input("Enter a positive number: "))
    print(" entered:", num)
    if num < 0:
        raise NegativeNumberError("Number cannot be negative.")

except NegativeNumberError as e:
    print("Custom Exception:", e)
```

```
 entered: -5
Custom Exception: Number cannot be negative.
```

2.Write a program that utilizes the else clause in a try-except block to perform cleanup tasks even if no exception occurs.

In [ ]:
```python
try:
    f = open("custom_exceptions.py", "r")
    text = f.read()
    print(text)
except FileNotFoundError:
    print("Error: File not found")
else:
    print("File read successfully")
    print(" closing file.")
    f.close()
```

```
class PythonException(Exception):
    pass

class NegativeNumberError(Exception):
    pass

class InvalidFilePathError(Exception):
    pass

class InsufficientFundsError(Exception):
    pass

class InvalidPasswordError(Exception):
    pass

File read successfully.
Performing cleanup: closing file.
```

3.Implement a program that uses nested try-except blocks to handle specific exceptions within multiple layers of function calls..

In [ ]:
```python
try:
    try:
        num1 = int(input("Enter first number: "))
        num2 = int(input("Enter second number: "))

        try:
            print(f"Result of {num1} / {num2} =", num1 / num2)
        except ZeroDivisionError:
            print("Inner: Cannot divide by zero.")
            raise         # sends the  ZeroDivisionError to the outer block.

    except ValueError:
        print("Outer: Invalid number.")
    except ZeroDivisionError:
        print("Outer: Zero division error ")
```

```
except Exception as e:   #catches exceptions that not handle by inner or outer bloc
    print("Top level:", e)
```

```
Inner: Cannot divide by zero.
Outer: Zero division error again.
```

4.Design a program that uses the finally clause to ensure resources are released regardless of exceptions or successful execution.

In [5]:
```
file = None

try:
    file_name = input("Enter file name to open: ")
    print(file_name)
    file = open(file_name, "r")
    print("File contents:")
    print(file.read())
except FileNotFoundError:
    print("Error: File not found.")
finally:
    if file is not None:
        file.close()
        print("File closed")
    print("finally block executed.")
```

```
sample.txt
Error: File not found.
finally block executed.
```

5.Write a program that uses exceptions to implement a retry mechanism for specific operations in case of temporary failures

In [17]:
```
correct_password = "dilip2910"
max_attempts = 3
attempt = 0

while attempt < max_attempts:
    try:
        print("Attempt:", attempt + 1)
        password = input("Enter password: ")
        print("entered password:",password)


        if password != correct_password:
            raise ValueError("Incorrect password. Please try again.")

        print("Login successful")
        break

    except ValueError as e:
        print(e)
        attempt += 1

if attempt == max_attempts:
    print("Login failed after", max_attempts, "attempts.")
```

```
Attempt: 1
entered password: aassd
Incorrect password. Please try again.
Attempt: 2
entered password: 232dfssdf
Incorrect password. Please try again.
Attempt: 3
entered password: dilip2910
Login successful
```

6.Write a Python program that prompts the user to enter a valid integer. Use a loop and exception handling to ensure that the user enters a valid integer, and print the entered integer.

In [9]:
```
while True:
    try:
        value = input("Enter a valid integer: ")
        print("entered ", value)
        num = int(value)

        break
    except ValueError:
        print("Invalid input try again.")
```

```
entered  d
Invalid input try again.
entered  i
Invalid input try again.
entered  2
```

7.Create a program that opens a file and reads its content. Handle the PermissionError exception and print a message if the program does not have permission to read the file.

In [3]:
```
file_name = input("Enter file name to read: ")
print(file_name)
try:
    with open(file_name, "r") as f:
        print("File opened successfully")
        print(f.read())
except PermissionError:
    print("Error: you dont have permission to read this file")
except FileNotFoundError:
    print("Error: File not found.")
```

```
adminonly.txt
Error: you dont have permission to read this file
```

8.Create a program that accepts user input for a list of numbers, and then calculates and prints the average of those numbers. Handle the ValueError exception when converting user input to numbers.

In [5]:
```
try:
    data = input("Enter integers separated by spaces: ")
```

```
    print("Given data",data)
    list = data.split()

    numbers = [int(x) for x in list]

    avg = sum(numbers) / len(numbers)
    print("Average is:", avg)

except ValueError:
    print("Error: Please enter only integers.")
```

```
Given data 1 2 3 4
Average is: 2.5
```

9.Create a program that asks the user to input two numbers and performs a mathematical operation (e.g., addition, subtraction, multiplication, division) based on user input Handle the potential exceptions that may arise during user input or calculation.

In [9]:
```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
    op = input("Enter operation (+, -, *, /): ")

    print(f"evaluating: {num1}{op}{num2}")
    if op == "+":
        result = num1 + num2
    elif op == "-":
        result = num1 - num2
    elif op == "*":
        result = num1 * num2
    elif op == "/":
        result = num1 / num2
    else:
        raise ValueError("Invalid operation.")

    print("Result:", result)

except ValueError as e:
    print("ValueError:", e)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
```

```
evaluating: 10/2
Result: 5.0
```

10.Create a program that simulates a login system. Ask the user to enter a username and password. Handle the KeyError when checking if the entered username exists in a dictionary and handle the ValueError when comparing the entered password..

In [ ]:
```
users = {
    "dilip": 1007,
    "harsh": 2910,
    "admin": 9999,
    "guest": 1111
```

```python
}

try:
    username = input("Enter username: ")
    print("given username",username)
    # May raise KeyError
    correct_pin = users[username]

    password = input("Enter numeric PIN password: ")
    print("given password",password)

    # May raise ValueError
    entered_pin = int(password)

    if entered_pin == correct_pin:
        print("Login successful")
    else:
        print("Incorrect password.")

except KeyError:
    print("Error: Username does not exist.")
except ValueError:
    print("Error: Password must be a number (integer).")
```

```
given username dilip
given password dilip2910
Error: Password must be a number (integer).
```